BOM

Browser Object Model

JavaScript Practical Workbook

Topics 15 – 22 · window · location · history · navigator · screen · dialogs · cookies · storage

window	location	history	navigator	screen	Dialogs	Cookies	Storage

TOPIC 15 window Object

The window object is the global object in browsers. It represents the browser window and provides properties and methods for controlling it.

Example 1 – window dimensions & location

```
// Window size
console.log(window.innerWidth);   // viewport width  e.g. 1366
console.log(window.innerHeight);  // viewport height e.g. 768

// Full screen size
console.log(window.screen.width);  // 1920
console.log(window.screen.height); // 1080

// Scroll position
console.log(window.scrollX, window.scrollY); // 0 0
// Output: 1366 768 1920 1080 0 0
```

Example 2 – window open & close

```
// Open a new tab/window
const newWin = window.open('https://example.com', '_blank',
                           'width=600,height=400');

// Close the opened window (only works on windows you opened)
setTimeout(() => newWin.close(), 3000);

// Focus / blur
newWin.focus();
newWin.blur();
// Output: Opens example.com in a 600×400 popup
```

Example 3 – Global variables on window

```
var globalVar = 'I am global';
console.log(window.globalVar); // I am global

function sayHi() { return 'Hi!'; }
console.log(window.sayHi());   // Hi!

// let/const are NOT on window
let secret = 42;
console.log(window.secret);    // undefined
// Output: I am global Hi! undefined
```

TOPIC 16 window.location

window.location provides information about the current URL and methods to navigate, reload, or redirect the browser.

Example 1 – Reading URL parts

```
// Assume URL: https://example.com:8080/path?q=JS#section

console.log(location.href);      // full URL
console.log(location.protocol);  // 'https:'
console.log(location.host);      // 'example.com:8080'
console.log(location.hostname);  // 'example.com'
console.log(location.port);      // '8080'
console.log(location.pathname);  // '/path'
console.log(location.search);    // '?q=JS'
console.log(location.hash);      // '#section'
// Output: Prints each URL component separately
```

Example 2 – Navigate & reload

```
// Redirect (adds to history)
location.href = 'https://google.com';

// Replace (no history entry)
location.replace('https://google.com');

// Reload current page
location.reload();

// Reload from server (not cache)
location.reload(true);
// Output: Page navigates / reloads accordingly
```

Example 3 – URL query string parsing

```
// URL: https://site.com/?name=Alice&age=25

const params = new URLSearchParams(location.search);

console.log(params.get('name'));  // Alice
console.log(params.get('age'));   // 25

params.forEach((val, key) => {
  console.log(key, '=', val);
});
// Output: Alice 25 name = Alice age = 25
```

TOPIC 17 window.history

window.history lets you navigate the browser's session history — moving forward, backward, or to a specific point — without reloading the page.

Example 1 – back, forward, go

```
// Go back one page
history.back();

// Go forward one page
history.forward();

// Jump by n steps (negative = back, positive = forward)
history.go(-2);  // go 2 pages back
history.go(1);   // go 1 page forward

// Total entries in history stack
console.log(history.length);  // e.g. 5
// Output: Navigates browser history stack
```

Example 2 – pushState (SPA navigation)

```
// Push a new state without reloading
history.pushState({ page: 'home' }, 'Home', '/home');
history.pushState({ page: 'about' }, 'About', '/about');

console.log(location.pathname);  // /about
console.log(history.length);     // increased by 2
// Output: /about (length increased)
```

Example 3 – replaceState & popstate event

```
// Replace current state (no new history entry)
history.replaceState({ page: 'profile' }, 'Profile', '/profile');

// Listen for back/forward navigation
window.addEventListener('popstate', (e) => {
  console.log('Navigated to state:', e.state);
  console.log('Current path:', location.pathname);
});
// Output: Navigated to state: {page:'home'}
```

TOPIC 18 window.navigator

window.navigator contains information about the browser and the user's device — browser name, OS, online status, geolocation, and more.

Example 1 – Browser & OS info

```
console.log(navigator.appName);     // Netscape (all modern browsers)
console.log(navigator.appVersion);  // browser version string
console.log(navigator.userAgent);   // full UA string
console.log(navigator.language);    // 'en-US'
console.log(navigator.platform);    // 'Win32' / 'MacIntel' etc.
console.log(navigator.onLine);      // true / false
// Output: Netscape en-US Win32 true
```

Example 2 – Geolocation API

```
if ('geolocation' in navigator) {
  navigator.geolocation.getCurrentPosition(
    (pos) => {
      console.log('Lat:', pos.coords.latitude);
      console.log('Lon:', pos.coords.longitude);
    },
    (err) => console.log('Error:', err.message)
  );
} else {
  console.log('Geolocation not supported');
}
// Output: Lat: 19.076 Lon: 72.877
```

Example 3 – Clipboard & online/offline

```
// Copy text to clipboard
navigator.clipboard.writeText('Hello World!')
  .then(() => console.log('Copied!'));

// Detect online/offline status
window.addEventListener('online',  () => console.log('Back online'));
window.addEventListener('offline', () => console.log('Gone offline'));

console.log('Online?', navigator.onLine); // true
// Output: Copied! Online? true
```

TOPIC 19 window.screen

window.screen provides information about the user's physical display screen — dimensions, color depth, and available space.

Example 1 – Screen dimensions

```
console.log(screen.width);        // total screen width  e.g. 1920
console.log(screen.height);       // total screen height e.g. 1080

// Available area (excluding taskbar/dock)
console.log(screen.availWidth);   // e.g. 1920
console.log(screen.availHeight);  // e.g. 1040  (taskbar ~40px)
// Output: 1920 1080 1920 1040
```

Example 2 – Color depth & pixel ratio

```
console.log(screen.colorDepth);        // 24 (bits per pixel)
console.log(screen.pixelDepth);        // 24
console.log(window.devicePixelRatio);  // 1 (or 2 on Retina)

// Detect high-DPI / Retina display
if (window.devicePixelRatio >= 2) {
  console.log('High-DPI display detected');
} else {
  console.log('Standard display');
}
// Output: 24 24 1 Standard display
```

Example 3 – Responsive screen check

```
function getDeviceType() {
  const w = screen.width;
  if (w <= 480)  return 'Mobile';
  if (w <= 1024) return 'Tablet';
  return 'Desktop';
}

console.log(getDeviceType());  // Desktop

// Center a popup on screen
const left = (screen.width  - 400) / 2;
const top  = (screen.height - 300) / 2;
window.open('page.html','_blank',
  `width=400,height=300,left=${left},top=${top}`);
// Output: Desktop
```

TOPIC 20 Dialogs & Timers

The BOM provides three dialog boxes (alert, confirm, prompt) and timer functions (setTimeout, setInterval, clearTimeout, clearInterval) for delayed and repeated execution.

Example 1 – alert, confirm, prompt

```
// Alert – shows a message
alert('Welcome to JavaScript BOM!');

// Confirm – returns true/false
const result = confirm('Do you want to continue?');
console.log(result);  // true or false

// Prompt – returns user input string
const name = prompt('Enter your name:', 'Guest');
console.log('Hello,', name);
// Output: Hello, Alice (user typed Alice)
```

Example 2 – setTimeout & clearTimeout

```
// Run once after 2 seconds
const id = setTimeout(() => {
  console.log('Runs after 2s');
}, 2000);

// Cancel before it fires
clearTimeout(id);  // cancels the above

// Chained timeouts (simulate interval)
function tick(n) {
  if (n <= 0) return;
  console.log('Tick', n);
  setTimeout(() => tick(n - 1), 1000);
}
tick(3);
// Output: Tick 3 Tick 2 Tick 1
```

Example 3 – setInterval & clearInterval

```
let count = 0;

const timer = setInterval(() => {
  count++;
  console.log('Count:', count);

  if (count === 5) {
    clearInterval(timer);  // stop after 5
    console.log('Timer stopped');
  }
}, 1000);
```

```
// Output: Count:1 Count:2 ... Count:5 Timer stopped
```

TOPIC 21 Cookies

Cookies are small key-value pairs stored in the browser. They persist across sessions and are sent with every HTTP request to the same domain.

Example 1 – Set & read cookies

```
// Set a cookie
document.cookie = 'username=Alice';
document.cookie = 'theme=dark; max-age=86400'; // 1 day

// Read all cookies (returns one string)
console.log(document.cookie);
// 'username=Alice; theme=dark'
// Output: username=Alice; theme=dark
```

Example 2 – Cookie with expiry & path

```
function setCookie(name, value, days) {
  const expires = new Date();
  expires.setDate(expires.getDate() + days);
  document.cookie = `${name}=${value}; `
    + `expires=${expires.toUTCString()}; path=/`;
}

function getCookie(name) {
  return document.cookie.split('; ')
    .find(c => c.startsWith(name + '='))
    ?.split('=')[1] ?? null;
}

setCookie('lang', 'en', 7);
console.log(getCookie('lang'));  // en
// Output: en
```

Example 3 – Delete a cookie

```
function deleteCookie(name) {
  // Set expiry to past date
  document.cookie = `${name}=; `
    + `expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/`;
}

setCookie('session', 'abc123', 1);
console.log(getCookie('session')); // abc123

deleteCookie('session');
console.log(getCookie('session')); // null
// Output: abc123 null
```

TOPIC 22 localStorage & sessionStorage

Web Storage provides two mechanisms: localStorage (persists until manually cleared) and sessionStorage (cleared when the tab closes). Both store string key-value pairs.

Example 1 – localStorage basics

```
// Store data
localStorage.setItem('name', 'Priya');
localStorage.setItem('score', '95');

// Read data
console.log(localStorage.getItem('name'));   // Priya
console.log(localStorage.getItem('score'));  // '95' (string!)

// Remove one key
localStorage.removeItem('score');

// Clear all
// localStorage.clear();
// Output: Priya 95
```

Example 2 – Storing objects (JSON)

```
const user = { name: 'Raj', age: 22, role: 'admin' };

// Serialize to JSON string
localStorage.setItem('user', JSON.stringify(user));

// Parse back to object
const stored = JSON.parse(localStorage.getItem('user'));
console.log(stored.name);  // Raj
console.log(stored.role);  // admin
// Output: Raj admin
```

Example 3 – sessionStorage vs localStorage

```
// sessionStorage – cleared when tab closes
sessionStorage.setItem('token', 'xyz789');
console.log(sessionStorage.getItem('token')); // xyz789

// Both have same API
console.log(localStorage.length);    // no. of LS keys
console.log(sessionStorage.length);  // no. of SS keys

// Iterate all localStorage keys
for (let i = 0; i < localStorage.length; i++) {
  const key = localStorage.key(i);
  console.log(key, ':', localStorage.getItem(key));
}
// Output: xyz789 user : {name:Raj,...}
```